

TASK1

TASK 1

Task 1.A

L1 D cache size is 192/6 i.e. 32 KB per instance.

1

Loop unrolling should be used to reduce loop overhead, such as branch instructions and index updates, as reducing these operations can allow more independent instructions to be executed in parallel and improve instruction-level parallelism.

Loop reordering should be used to improve spatial locality by keeping contiguous memory accesses in the innermost loop, as this can make better use of cache lines and reduce the overhead of inefficient memory accesses during convolution

2

1. Balanced unrolling factor should be applied for loop unrolling as a very high unrolling factor may put pressure on registers and may pollute the cache and a very low unrolling factor may not utilize the memory bandwidth to the fullest for an optimum performance .
2. The loop order should be selected carefully as accessing contiguous memory locations in the innermost loop can improve spatial locality and cache utilization, while an unsuitable loop order may result in inefficient memory accesses and reduce performance.

3

Matrix_Size	Unroll	Reorder
128	1.17	1.03
256	1.12	0.9
512	1.1	1.09
1024	1.05	1.02
2048	1.01	0.98

In the naive implementation, each output pixel is computed by accumulating all K^2 kernel operations in a register and writing the result only once, while the reordered implementation For a fixed (ky,kx), the reordered loop sweeps the entire (H xW) image before moving to the next kernel tap, so the same out(oy,ox) element is accessed again only after a large amount of unrelated data has been accessed. For a (2048 x 2048) image, this corresponds to approximately 16 MB of data between two consecutive accesses, which is much larger than the available cache capacity. Therefore, the data is unlikely to remain in the cache, resulting in poor cache reuse and increasing the memory-access cost for both input and output data. This is the reason reorder can be as slow or slower than naive

Task 1.B

Deliverable 1

L1 D cache size = 32 KB

```
abhi1kshil@abhi1kshil-Victus-by-HP-Laptop-16-e0xxx:~/Desktop/pai-dhurandhar-microarchitecture-main/task1$ perf stat -d ./bin/conv naive
CS683 PA-1 Task 1 2D convolution optimization harness

=== Workload H=2048 W=2048 K=3 seed=1234 (not graded) ===
stage      correct  time(ms)  GFLOP/s  speedup
-----
naive (ref)  yes        32.970    2.29     1.00x

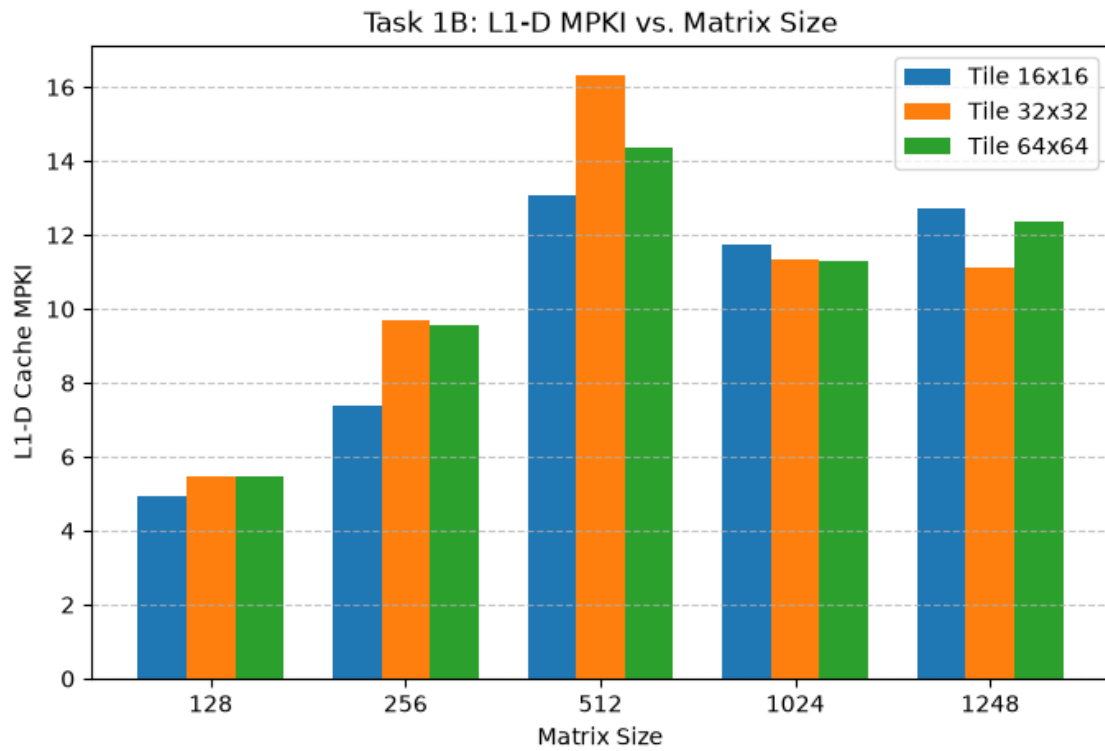
Performance counter stats for './bin/conv naive':

      20      context-switches          #    41.5 cs/sec  cs_per_second
         0      cpu-migrations          #         0.0 migrations/sec  migrations_per_second
    16,538      page-faults            # 34334.5 faults/sec  page_faults_per_second
    481.67 msec task-clock              #     1.0 CPUs    CPUs_utilized
  15,850,274      L1-dcache-load-misses      #    1.6 %    lid_miss_rate      (42.67%)
   2,610,693      branch-misses            #    0.4 %    branch_miss_rate    (42.68%)
  648,310,349      branches                # 1346.0 M/sec    branch_frequency  (42.87%)
   998,061,502      cpu-cycles              #    2.1 GHz    cycles_frequency  (42.92%)
  4,570,949,321      instructions            #    4.6 instructions    insn_per_cycle  (43.04%)
  38,688,580      stalled-cycles-frontend      #    0.04 frontend_cycles_idle  (43.02%)

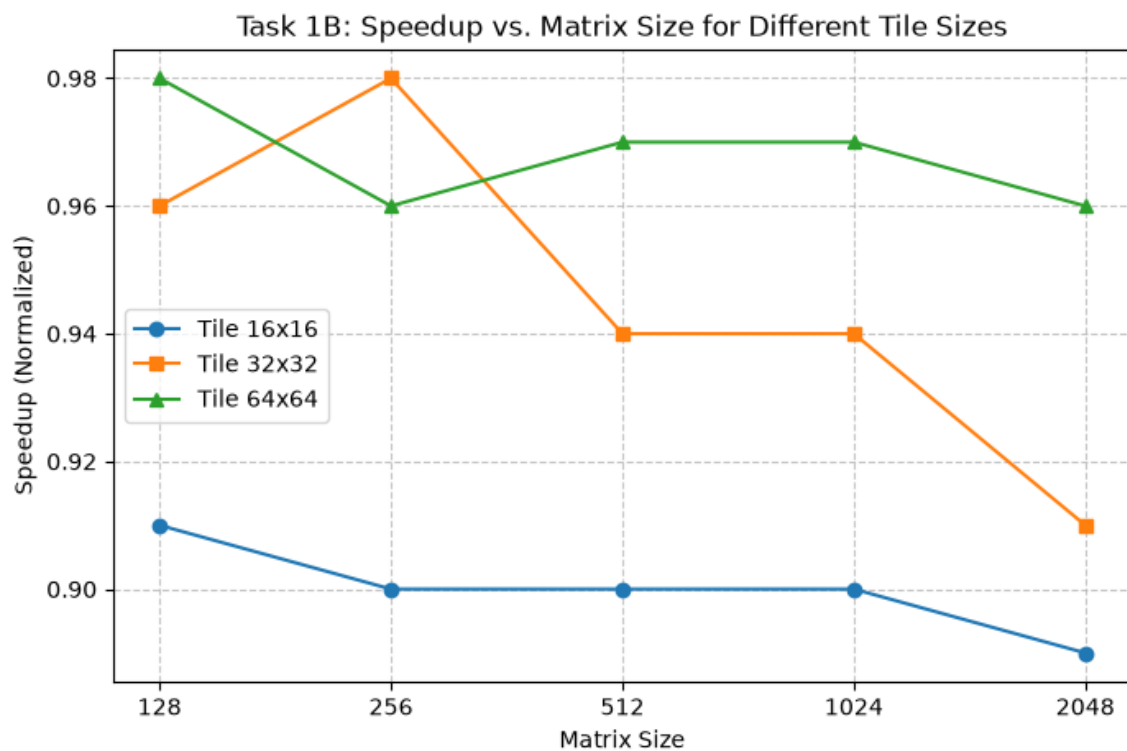
0.484706015 seconds time elapsed

0.420786000 seconds user
0.063117000 seconds sys
```

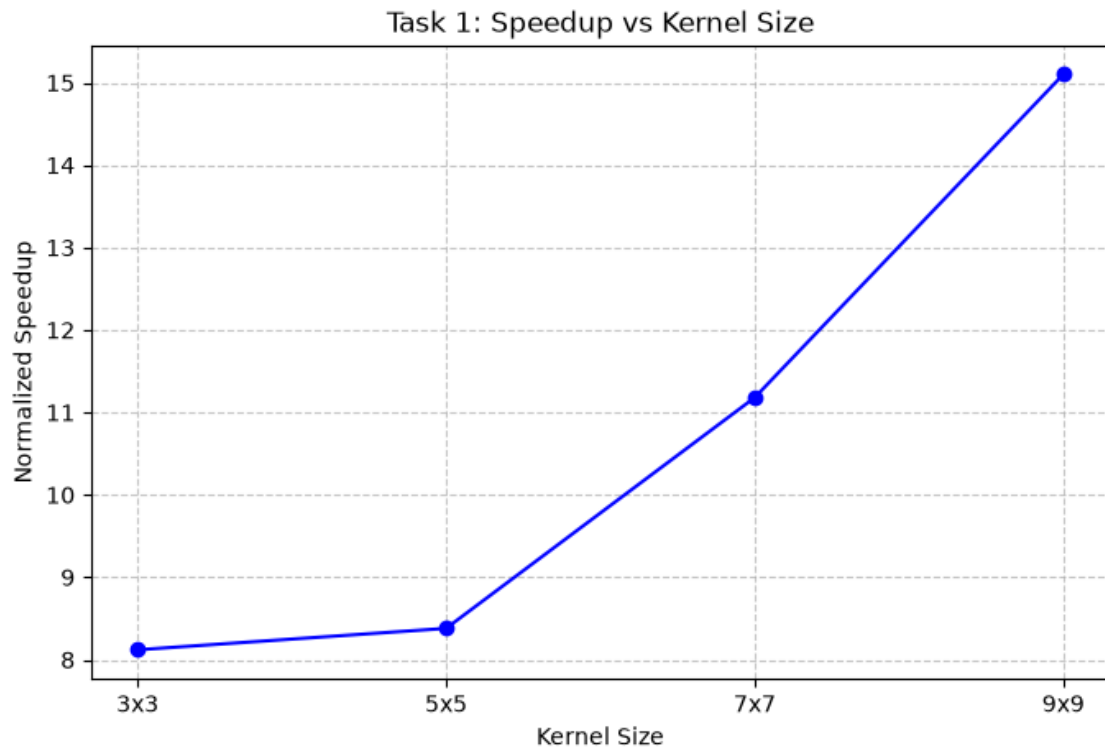
Deliverable 2 & 3



Plot of L1-D MPKI vs Matrix size of Input for different tile size



Plot of Speedup vs Matrix size of Input for different tile size



Plot of Speedup vs Kernel size of Input for a 2048x2048 input matrix

1

The observed values are:

Metric	Naive	Tiled
Number of Instructions	4,570,949,321	7,695,482,316
Number of Instructions	15,858,274	67,191,980
L1-D Cache Misses	3.46	8.73

2

According to the data, MPKI rises at larger tile sizes. As tile size grows, the working set approaches or exceeds L1-D cache capacity. This introduces capacity and conflict evictions between neighboring matrix rows inside the L1 cache, causing L1-D load misses to rise and resulting in higher MPKI values.

For small matrix size The entire matrix working set fits across L1/L2 caches, producing a low baseline MPKI. For medium matrix size the working set spills out of the L2 cache into L3, causing MPKI to peak across all tile sizes. For large N values execution becomes completely memory-bandwidth bound. Hardware prefetchers engage effectively on the continuous tile streams, stabilizing MPKI.

3

The Matrix size vs tile size speed up ratios

Matrix_Size	Tile_16	Tile_32	Tile_64
128	0.91	0.96	0.98
256	0.9	0.98	0.96
512	0.9	0.94	0.97
1024	0.9	0.94	0.97
2048	0.89	0.91	0.96

No Speed up was not achieved for any given combination of tile and matrix size.

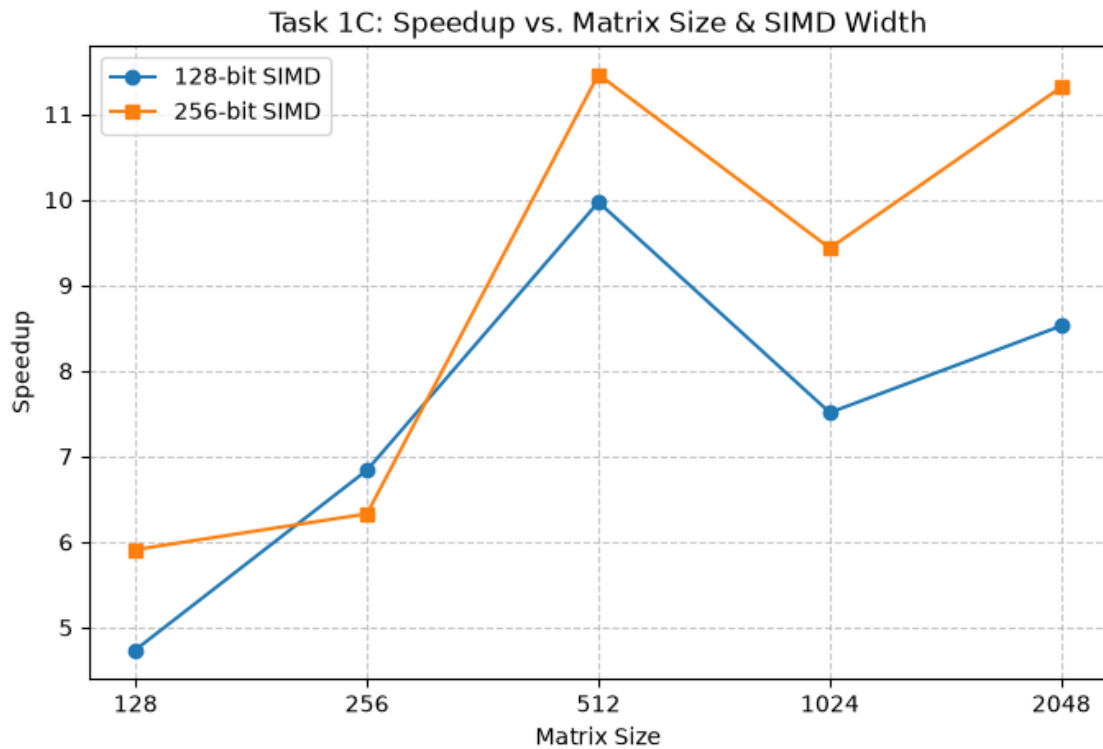
Justification

Metric	Tiny Tile ($T = 16$)	Medium Tile ($T = 32$)	Large Tile ($T = 64$)
Output Elements Computed (T^2)	256	1,024	4,096
Input Elements Fetched ($(T + 2)^2$)	324	1,156	4,356
Extra Elements Fetched ($4T + 4$)	68	132	260
Data Overhead Percentage	26.56%	12.89%	6.35%
Total Tile Count (2048×2048)	16,384	4,096	1,024

Impact

Redundant Data Loading For $T = 16$, 26.56% of total memory traffic consists of redundant fetches. With increase in the tile size the extra re-fetches decrease by an order of $1/T$ where T is the tile size , so greater the tile size lesser is the percentage extra re-fetches.

Task 1.C



Plot of Speedup vs Matrix sizes for different Register width

1

Matrix Size	Naive Instructions	Naive L1D Misses	Naive L2 Misses	SIMD Instructions	SIMD L1D Misses	
128	20,884,642	90,372	17,177	7,963,580	90,697	
256	74,461,608	227,263	28,049	22,541,219	226,575	
512	287,750,232	771,861	49,583	80,220,847	772,963	
1024	1,141,597,366	3,104,103	169,068	310,977,316	3,098,193	1
2048	4,556,540,732	15,940,233	522,131	1,236,427,018	14,321,332	5

Moving to AVX2 single-precision SIMD intrinsics allows the CPU to process 8 floating-point numbers simultaneously in a single 256-bit register reducing inner loop execution iterations by a factor of 8x and directly cutting down on scalar arithmetic instructions and inner-loop branching overhead. The SIMD implementation replaces these with `mm256_fmadd_ps`, which performs 8 multiplications and 8 additions in a single hardware instruction, halving the instruction count for core arithmetic operations. `mm256_set1_ps` duplicates a single 32-bit filter weight across all eight vector lanes of a 256-bit register. Pre-

broadcasting these weights into the `vec[KxK]` array before entering the spatial image loops hoists the weight setup entirely out of the core computation path.

2

The SIMD implementation achieved a speedup of approximately 6×–7× compared with the baseline implementation. The main reason for this improvement is the use of data-level parallelism, where multiple independent operations are performed simultaneously using SIMD instructions. The major contributing factors are:

- The 256-bit AVX2 registers can hold eight single-precision floating-point values, allowing the processor to perform the same operation on eight elements simultaneously. This reduces the number of iterations required in the inner convolution loop and increases computational throughput.
- The `mm256_fmadd_ps` instruction performs multiplication and addition together on all eight vector lanes. Thus, one instruction performs eight multiplications and eight additions, reducing the total number of instructions and improving the utilization of the processor's execution units.

3

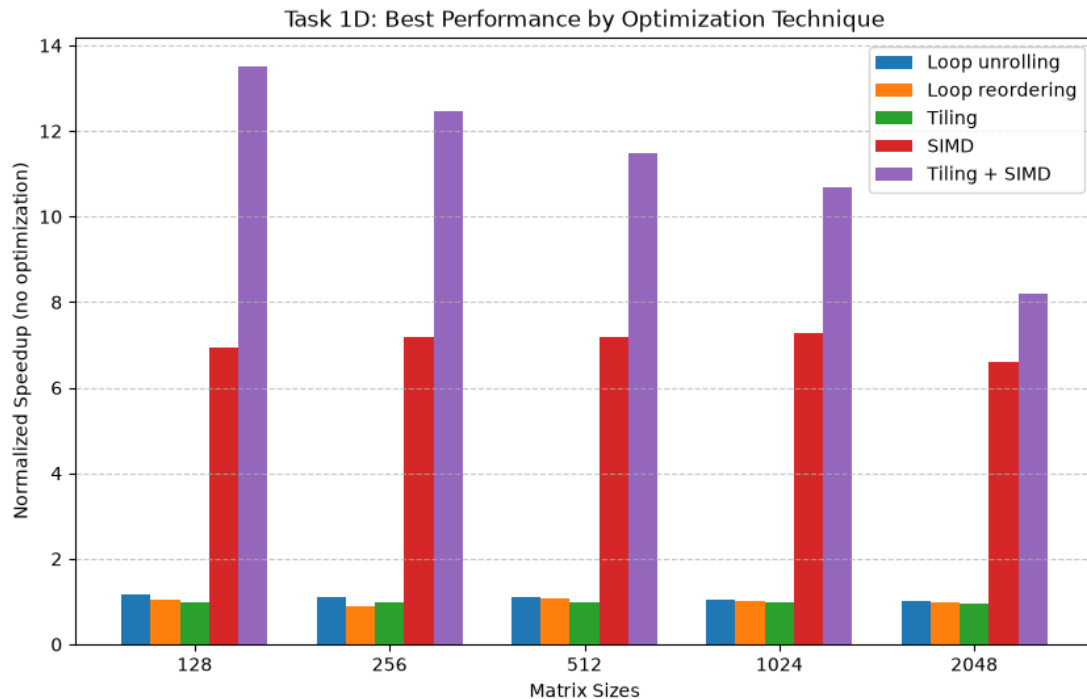
1. SIMD Intrinsic s

Intrinsic	Purpose
<code>_mm256_set1_ps</code>	Pre-broadcasts scalar filter weights across vector registers.
<code>_mm256_setzero_ps</code>	Clears accumulation registers and fills with 0.0f.
<code>_mm256_loadu_ps</code>	Ingests 8 contiguous single-precision floats into a 256-bit register.
<code>_mm256_fmadd_ps</code>	Executes 8 parallel fused multiply-accumulate operations ($a \times b + c$) in one hardware cycle .
<code>_mm256_storeu_ps</code>	Writes 8 computed output results from a vector register back to DRAM/cache.

`mm256_set1_ps` broadcasts one kernel weight to all 8 lanes, so the same weight can be applied to 8 pixels together, while **`mm256_setzero_ps`** initializes the accumulator efficiently. **`mm256_loadu_ps`** and **`mm256_storeu_ps`** are used because the input and output addresses may not be 32-byte aligned, avoiding

the need for extra alignment operations. **mm256_fmadd_ps** performs multiplication and addition together for all 8 lanes, reducing the number of instructions compared with separate multiply and add operations

Task 1.D



The combined Tiling + SIMD implementation provides the best overall performance across the different matrix sizes, outperforming the individual optimization techniques. Tiling improves cache locality by keeping frequently reused data within the cache, while SIMD processes multiple elements simultaneously using vector instructions.